Rendimiento Computacional

Prof. Ramiro Andrés Barrios Valencia

Definición de Rendimiento

Existen dos formas de medir el rendimiento:

Tiempo de respuesta:

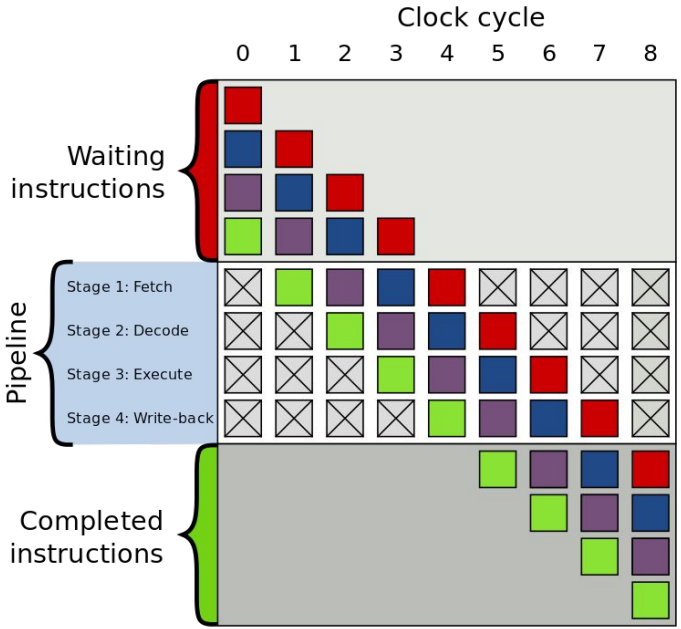
Tiempo entre el comienzo y finalización de una tarea, llamado tiempo de ejecución. Medida que favorece al usuario

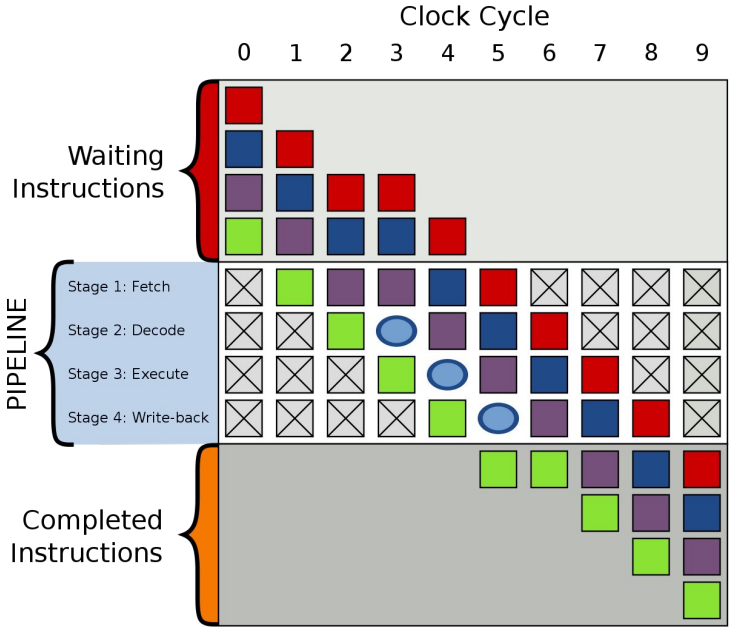
Productividad (Throughput):

Cantidad de trabajo realizado en un tiempo determinado. Medida que favorece al sistema.

Tiempo de respuesta Vs Productividad

Caso Pipeline:





Tiempo de respuesta Vs Productividad

Caso Pipeline:

	Tiempo de respuesta	Productividad
Reloj ⇑	⇓	⇑
Múltiples procesadores. Tareas separadas	-	⇑
Procesador paralelo	⇓	⇑
Segmentación	-	⇑

Rendimiento y tiempo de ejecución

- El rendimiento de un computador X es inversamente proporcional al tiempo de ejecución:

$$\mathrm{Re}\,n\,\mathrm{dim}\,iento_X = \frac{1}{Tiempo\ de\ ejecución_X}$$

- Computador X es n% más rápido que el computador Y

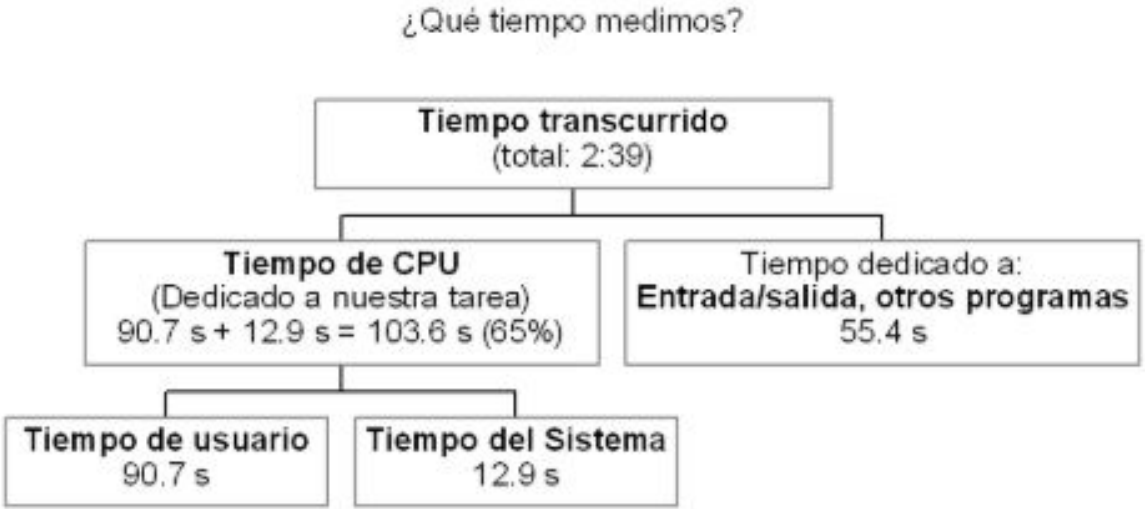
$$\frac{\mathrm{Re}\,n\,\mathrm{dim}\,iento_X - \mathrm{Re}\,n\,\mathrm{dim}\,iento_Y}{\mathrm{Re}\,n\,\mathrm{dim}\,iento_Y} * 100 = n$$

- Computador Y es n% más lento que el computador X

$$\frac{\mathrm{Re}\,n\,\mathrm{dim}\,iento_X - \mathrm{Re}\,n\,\mathrm{dim}\,iento_Y}{\mathrm{Re}\,n\,\mathrm{dim}\,iento_X} * 100 = n$$

Medidas de tiempo

- Tiempo (segundos)
  - Única medida completa y fiable del rendimiento de un computador.
- Comando Unix: *time "programa" --> 90.7 u 12.9 s 2:39 65%*



Rendimiento de la CPU - Ecuación clásica (1)

$$T_{CPU} = \text{Nº ciclos reloj por programa} \times \text{Tiempo de ciclo}$$

$$CPI_{MEDIO} = \frac{\text{Nº ciclos de reloj por programa}}{\text{Recuento de instrucciones}}$$

(CPI_{MEDIO}: Nos permite comparar dos implementaciones de una misma arquitectura)

$$T_{CPU} = \text{Recuento de instrucciones} \times CPI_{MEDIO} \times \text{Tiempo de ciclo}$$

$$T_{CPU} = N * CPI * T_C$$

Rendimiento de la CPU - Ecuación clásica (2)

Análisis de dependencias:

$$\text{Tiempo_CPU} = \frac{\text{Segundos}}{\text{Programa}} = \frac{\text{Instruciones}}{\text{Programa}} \times \frac{\text{Ciclos}}{\text{Instrucción}} \times \frac{\text{Segundos}}{\text{Ciclo}}$$

$$T = N \times CPI \times T_c$$

	Recuento de Instrucciones	CPI	Tiempo de Ciclo
Programa	x		
Compilador	x	(x)	
Repertorio de instrucciones	x	x	
Organización-Estructura		x	x
Tecnología			x

Métricas populares de rendimiento - MIPS

- MIPS (millones de instrucciones por segundo)

$$MIPS_{Nativos} = \frac{\text{Frecuencia del reloj}}{CPI \times 10^6}$$

$$MIPS_{Relativos} = \frac{Tiempo_{Referencia}}{Tiempo_{No_estimado}} \times MIPS_{Referencia}$$

- Inconvenientes de la métrica:
  - Los MIPS dependen del repertorio de instrucciones. No podemos comparar distintas arquitecturas.
  - Los MIPS varían entre programas en el mismo computador.
  - Los MIPS pueden variar inversamente al rendimiento.

Métricas populares de rendimiento - MFLOPS

- MFLOPS (Millones de operaciones en punto flotante por segundo)

$$\text{MFLOPS} = \frac{\text{Nº operaciones en punto flotante}}{\text{Tiempo de ejecución} \cdot 10^6}$$

- Inconvenientes:
  - Sólo aplicable a las operaciones en punto flotante.
  - Instrucciones en punto flotante no comparables entre diferentes máquinas.

    Máquina A : *, -, * y / **Máquina B** : No tiene "/" (dará mas MFLOPS)
  - Mezcla de operaciones rápidas y lentas.
    - Los MFLOPS para 100 *sumas* serán mayores que para 100 *divisiones*.
- MFLOPS normalizados:

Operaciones	Pesos
+, - , * y comparar	1
÷, raíz cuadrada	4
Exp, seno	8

Benchmark (I)

Programas para la evaluación del rendimiento. Programas que forman una carga de trabajo con la que le usuarios espera predecir el rendimiento de la carga de trabajo real.

Existen algunas variaciones:

Programas reales típicos: compiladores (gcc), tratamiento de texto (TeX), herramientas CAD (Spice).

Núcleos obtenidos de programas reales: pequeñas partes intensivas en el tiempo de programas reales. Ej: Livermore loops y Linpack

Benchmark (II)

Existen algunas variaciones:

Programas triviales o benchmarks reducidos (toys): 10 - 100 líneas de programa. Resultado conocido antes de ejecutarlo. Ej. Quicksort, puzzle,etc.

Programas sintéticos: creados artificialmente. No resuelven un problema real ni están basados en la solución a un problema real. Ej: Whetstone, Dhrystone, etc.

Consideraciones finales

- Los benchmark reducidos (toys) y los sintéticos no cargan la memoria principal del sistema de forma realista (todo el programa cabe en la memoria caché)
- Una vez que el benchmark se estandariza inmediatamente aparecen mejoras específicas para el mismo, elevando los resultados de rendimiento.
- Los benchmark reales son difíciles de realizar. Benchmark no portables, compiladores no disponibles.

https://haydenjames.io/linux-benchmark-scripts-tools/